Actividad 3. Evaluar la seguridad mediante herramientas automáticas

Datos de los Estudiantes

Nombre & Apellidos: ENDIKA PRADERA TOUZANI, RUBEN DOUGALL VAN KOMMER

Detalles del Curso

Nombre de la Asignatura: METODOLOGÍAS DE DESARROLLO SEGURO

Nombre del Profesor: SERGIO IGLESIAS PEREZ

ÍNDICE

1. PROYECTO..... 3

2. OBJETIVO..... 3

3. HERRAMIENTAS Y TÉCNICAS UTILIZADAS Y JUSTIFICACIÓN..... 3

4. ALTERACIONES REALIZADAS PARA GENERAR ALERTAS..... 3

5. EJECUCIÓN DE LAS PRUEBAS..... 4

5.1. Preparación del entorno..... 4

5.2. Ejecución por técnica..... 4

6. RESULTADOS OBTENIDOS..... 5

7. ALTERACIONES/ESCENARIOS DE PRUEBA DOCUMENTADOS..... 5

8. PLAN DE REMEDIACIÓN..... 6

9. EVIDENCIAS..... 6

10. CONCLUSIÓN..... 11

1. PROYECTO

FollowMe Smart Parking (continuación directa de ACT.2).
Aplicación desplegada en contenedores Docker compuesta por backend, frontend y AI engine.

2. OBJETIVO

Evaluar la seguridad del proyecto mediante el uso de herramientas automáticas, identificando vulnerabilidades en el código, dependencias, configuración y el comportamiento en ejecución.

3. HERRAMIENTAS Y TÉCNICAS UTILIZADAS Y JUSTIFICACIÓN

1) SAST (Semgrep)
    - Permite analizar el código fuente (backend/frontend) sin ejecutarlo para detectar patrones inseguros y malas prácticas.
2) SCA (npm audit, pip-audit)
    - Identifica vulnerabilidades conocidas (CVEs) en librerías de terceros utilizadas por la aplicación.
3) Secret Scanning (Gitleaks)
    - Detecta posibles credenciales, tokens o claves expuestas en el repositorio.
4) Container/Image Scan (Trivy)
    - Evalúa vulnerabilidades en imágenes Docker y dependencias del sistema.
5) DAST básico (pruebas HTTP)
    - Evalúa el comportamiento de la aplicación en ejecución.

4. ALTERACIONES REALIZADAS PARA GENERAR ALERTAS

- Introducción de una dependencia vulnerable en el entorno de pruebas.
- Simular de una credencial hardcodeada en un archivo temporal para validar detector de secretos.
- Prueba de un endpoint con payload malicioso (XSS/inyección) para validar los filtros y las respuestas.
- Exposición de una configuración no recomendada en entorno local para comprobar alertas de hardening.

Para poder generar alertas, fue necesario realizar una serie de modificaciones. Estas acciones sólo deben llevarse a cabo con fines de prueba y deben revertirse antes de pasar a producción.

5. EJECUCIÓN DE LAS PRUEBAS

5.1. Preparación del entorno

1) Levantar entorno:
   - `docker compose up -d`
2) Verificar servicios:
   - `docker compose ps`
   - `curl http://localhost:3000/health`
   - `curl http://localhost:8001/health`

5.2. Ejecución por técnica

1) SAST (Semgrep)
   - Instalar Semgrep:
     - `pipx install semgrep` o `pip install semgrep`
   - Ejecución sobre el código del proyecto:
     - `semgrep --config auto followme/`
   - Generación de reporte en JSON:
     - `semgrep --config auto followme/ --json > reports/semgrep.json`
2) SCA (npm audit / pip-audit)
   - Backend:
     - `cd followme/backend && npm audit --json > ../reports/npm_audit_backend.json`
   - Frontend:
     - `cd ../web && npm audit --json > ../reports/npm_audit_web.json`
   - IA (Python):
     - `pip-audit -f json > followme/reports/pip_audit.json`
3) Secret Scanning (Gitleaks)
   - Ejecución sobre el repositorio:
     - `gitleaks detect -s . -r followme/reports/gitleaks.json`
   - Generación de reporte en JSON
4) Container Scan (Trivy)
   - Escaneo de imágenes:
     - `trivy image followme-backend > followme/reports/trivy_backend.txt`
     - `trivy image followme-web > followme/reports/trivy_web.txt`
     - `trivy image followme-ai-engine > followme/reports/trivy_ai.txt`
   - Resultados exportados en texto

5) DAST básico
    - Con app levantada, ejecutar pruebas sobre endpoints críticos (auth, admin, eventos).
    - Validar respuestas esperadas (401/403/400) y ausencia de exposición sensible.
    - Guardar capturas y logs en carpeta evidencias/.

6. RESULTADOS OBTENIDOS

1) DAST (servicios en ejecución)
    - Backend: HTTP 200 - Request procesada correctamente
    - AI Engine: HTTP 200 - Servicio operativo
    - Frontend: HTTP 200 - Superficie web disponible
2) SAST (Semgrep)
   Se detectaron hallazgos relevantes de seguridad, entre ellos:
    - Contenedores ejecutando como root (falta `USER` no privilegiado en Dockerfiles).
    - Riesgos de path traversal en controlador de eventos.
    - Recomendaciones de hardening en `docker-compose.yml` (`no-new-privileges`, `read_only`).
    - Referencia a OWASP en las reglas reportadas (misconfiguración y diseño inseguro).
3) SCA (npm audit)
    - Detección de dependencias vulnerables en backend y frontend.
    - Generación de reportes JSON para análisis y priorización.
4) Secret Scanning
    - Detección positiva de credencial de prueba introducida de forma controlada.
5) Container Scan
    - Identificación de CVEs en imágenes base y paquetes del sistema, algunos de ellos pendientes de parche.

7. ALTERACIONES/ESCENARIOS DE PRUEBA DOCUMENTADOS

La evaluación se orientó a detectar misconfiguraciones, dependencias con riesgo y exposición de superficie en contenedores, manteniendo un entorno de práctica controlado para observar alertas y evidencias automáticas.

Para ello, se trabajó con los siguientes escenarios:

- Introducción de una dependencia vulnerable.
- Simulación de una credencial hardcodeada.
- Pruebas con payloads maliciosos en endpoints.
- Configuración no recomendada en entorno local.

8. PLAN DE REMEDIACIÓN

1) Ejecutar los servicios con usuario no root en los tres Dockerfiles.
2) Endurecer la configuración de `docker-compose.yml` mediante `security_opt: ["no-new-privileges:true"]` y `read_only` donde aplique.
3) Revisar y sanear rutas de archivos en el controlador de eventos para prevenir ataques de path traversal.
4) Actualizar dependencias vulnerables detectadas por `npm audit`, priorizando aquellas de severidad crítica y alta.
5) Integrar escaneos automáticos en CI/CD (SAST, SCA, análisis de contenedores) en cada pull request.

9. EVIDENCIAS

Los resultados de las pruebas realizadas se encuentran documentados mediante los siguientes reportes técnicos en la carpeta `reports/`:

- Ejecución de servicios (Docker):
  - `reports/00_docker_compose_ps.txt`
    - `followme_ai_engine` levantado en puerto 8001.
    - `followme_backend` levantado en puerto 3000.
    - `followme_db` levantado y healthy.
    - `followme_web` levantado en puerto 5173.

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
followme_ai_engine	followme-ai-engine	"uvicorn app.main:ap…"	ai-engine	15 seconds ago	Up Less than a second (health: starting)	0.0.0.0:8001->8001/tcp, [::]:8001->8001/tcp
followme_backend	followme-backend	"docker-entrypoint.s…"	backend	15 seconds ago	Up Less than a second	0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
followme_db	postgres:15-alpine	"docker-entrypoint.s…"	db	15 seconds ago	Up 10 seconds (healthy)	0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp
followme_web	followme-web	"docker-entrypoint.s…"	web	13 seconds ago	Up Less than a second	0.0.0.0:5173->5173/tcp, [::]:5173->5173/tcp

- Verificación de estado de servicios (health checks):
  - `reports/01_backend_health.txt`

```
HTTP/1.1 200 OK
X-Powered-By: Express
Access-Control-Allow-Origin: *
Content-Type: application/json; charset=utf-8
Content-Length: 54
ETag: W/"36-KB48DsER8x3kU9x5GW7FGtMsie4"
Date: Sun, 19 Apr 2026 09:26:00 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"status":"OK","timestamp":"2026-04-19T09:26:00.742Z"}
```

    - El archivo "`reports/01_backend_health.txt`" muestra que el endpoint de salud del backend responde correctamente con un código HTTP 200 y representa un estado "`OK`".

- `reports/02_ai_health.txt`

```
HTTP/1.1 200 OK
date: Sun, 19 Apr 2026 09:26:00 GMT
server: uvicorn
content-length: 72
content-type: application/json

{"status":"healthy","version":"1.0.0","service":"FollowMe Smart Engine"}
```

  - Muestra que el servicio AI Engine responde correctamente con HTTP 200 y estado “`healthy`”, con lo cual tiene un funcionamiento correcto.

- `reports/03_web_head.txt`

```
HTTP/1.1 200 OK
Vary: Origin
Content-Type: text/html
Cache-Control: no-cache
Etag: W/"2e1-LYbXe44WyJmMDNYCb7H6CFvcbcA"
Date: Sun, 19 Apr 2026 09:26:00 GMT
Connection: keep-alive
Keep-Alive: timeout=5
Content-Length: 737

<!doctype html>
<html lang="es">
  <head>
    <script type="module">import { injectIntoGlobalHook } from "/@react-refresh";
injectIntoGlobalHook(window);
window.$RefreshReg$ = () => {};
window.$RefreshSig$ = () => (type) => type;</script>

    <script type="module" src="/@vite/client"></script>
```

  - El frontend responde correctamente con HTTP 200 y devuelve contenido HTML.

- Análisis de dependencias (SCA - npm audit):
  - `reports/04_npm_audit_backend.json`


```
"metadata": {
  "vulnerabilities": {
    "info": 0,
    "low": 0,
    "moderate": 2,
    "high": 4,
    "critical": 2,
    "total": 8
  },
```


    - Entre las dependencias afectadas aparecen: `"fast-xml-parser", "handlebars", "express-rate-limit", "minimatch", "path-to-regexp", "picomatch", "brace-expansion" y "qs".`
    - El reporte indica que existen correcciones disponibles, por lo que se recomienda actualizar las dependencias afectadas priorizando por su severidad.
  - `reports/05_npm_audit_web.json`


```
"metadata": {
  "vulnerabilities": {
    "info": 0,
    "low": 0,
    "moderate": 4,
    "high": 12,
    "critical": 0,
    "total": 16
  },
  "dependencies": {
    "prod": 14,
    "dev": 289,
    "optional": 46,
    "peer": 0,
    "peerOptional": 0,
    "total": 302
  }
```


    - Tras terminar el análisis se han identificado 16 vulnerabilidades en total, distribuidas en:
      - 12 de severidad alta
      - 4 severidad moderada

      No se detectaron vulnerabilidades críticas ni de bajo nivel.

- Análisis de Secret Scanning (Gitleaks):
    - Está herramienta tiene la capacidad de detectar credenciales expuestas para nuestro caso, se identificaron tokens tipo API key y cabeceras Authorization en archivos de prueba.

```
gitleaks detect --source . --verbose

  ┌──────────────────────────────────────────────────────────┐
  │   GITLEAKS - Secret Scanning · FollowMe Project          │
  │   Herramienta: gitleaks v8.x  |  Fecha: 04-May-2026      │
  └──────────────────────────────────────────────────────────┘


    ○
    │╲
    │ ○
    ○ ╲
    │   gitleaks

Finding:     curl http://localhost:5000/api/eventos

# Con token inválido - Debe fallar (401)
curl -H "Authorization: Bearer          □[1;3;mtoken-invalido□[0m" http://localhost:50...
Secret:      □[1;3;mtoken-invalido□[0m
RuleID:      curl-auth-header
Entropy:     3.378783
File:        followme/GUIA_TESTING.md
Line:        100
Commit:      546ecfef23aea195373b792a5f992aa78522645e
Author:      ENDIKA PRADERA TOUZANI
Email:       endikapraderatouzani@ENDIKAs-MacBook-Air.local
Date:        2026-01-07T20:49:50Z
Fingerprint: 546ecfef23aea195373b792a5f992aa78522645e:followme/GUIA_TESTING.md:curl-auth-header:100
Link:        https://github.com/endikapradera/followmeFinal/blob/546ecfef23aea195373b792a5f992aa78522645e/followme/GUIA_TES

Finding:     #   "token": "□[1;3;meyJhbGciOiJIUzI1NiIs...□[0m"
Secret:      □[1;3;meyJhbGciOiJIUzI1NiIs...□[0m
RuleID:      generic-api-key
Entropy:     3.816403
File:        followme/GUIA_TESTING.md
Line:        201
Commit:      546ecfef23aea195373b792a5f992aa78522645e
Author:      ENDIKA PRADERA TOUZANI
Email:       endikapraderatouzani@ENDIKAs-MacBook-Air.local
Date:        2026-01-07T20:49:50Z
Fingerprint: 546ecfef23aea195373b792a5f992aa78522645e:followme/GUIA_TESTING.md:generic-api-key:201
Link:        https://github.com/endikapradera/followmeFinal/blob/546ecfef23aea195373b792a5f992aa78522645e/followme/GUIA_TES

□[90m9:37PM□[0m □[32mINF□[0m □[1m7 commits scanned.□[0m
□[90m9:37PM□[0m □[32mINF□[0m □[1mscanned ~84426827 bytes (84.43 MB) in 5.24s□[0m
□[90m9:37PM□[0m □[33mWRN□[0m □[1mleaks found: 2□[0m
```

- Análisis de contenedores (Trivy):
    - Tras el análisis con Trivy, se identificaron los paquetes del sistema presentes en la imagen Docker, sin detectarse vulnerabilidades de severidad alta o crítica en esta ejecución.

```
trivy image followme-backend:latest --severity HIGH,CRITICAL --scanners vuln
```

```
TRIVY - Container Vulnerability Scan · followme-backend:latest
Severidad: HIGH, CRITICAL | Fecha: 04-May-2026
```

followme-backend:latest (alpine 3.23.3)	alpine	18
app/node_modules/.prisma/client/package.json	node-pkg	0
app/node_modules/@babel/code-frame/package.json	node-pkg	0
app/node_modules/@babel/compat-data/package.json	node-pkg	0
app/node_modules/@babel/core/node_modules/debug/package.json	node-pkg	0
app/node_modules/@babel/core/node_modules/ms/package.json	node-pkg	0
app/node_modules/@babel/core/node_modules/semver/package.json	node-pkg	0
app/node_modules/@babel/core/package.json	node-pkg	0
app/node_modules/@babel/generator/package.json	node-pkg	0
app/node_modules/@babel/helper-compilation-targets/node_modules/semver/package.-json	node-pkg	0
app/node_modules/@babel/helper-compilation-targets/package.json	node-pkg	0
app/node_modules/@babel/helper-globals/package.json	node-pkg	0
app/node_modules/@babel/helper-module-imports/package.json	node-pkg	0
app/node_modules/@babel/helper-module-transforms/package.json	node-pkg	0
app/node_modules/@babel/helper-plugin-utils/package.json	node-pkg	0
app/node_modules/@babel/helper-string-parser/package.json	node-pkg	0
app/node_modules/@babel/helper-validator-identifier/package.json	node-pkg	0
app/node_modules/@babel/helper-validator-option/package.json	node-pkg	0
app/node_modules/@babel/helpers/package.json	node-pkg	0
app/node_modules/@babel/parser/package.json	node-pkg	0
app/node_modules/@babel/plugin-syntax-async-generators/package.json	node-pkg	0
app/node_modules/@babel/plugin-syntax-bigint/package.json	node-pkg	0
app/node_modules/@babel/plugin-syntax-class-properties/package.json	node-pkg	0
app/node_modules/@babel/plugin-syntax-class-static-block/package.json	node-pkg	0
app/node_modules/@babel/plugin-syntax-import-attributes/package.json	node-pkg	0
app/node_modules/@babel/plugin-syntax-import-meta/package.json	node-pkg	0
app/node_modules/@babel/plugin-syntax-json-strings/package.json	node-pkg	0
app/node_modules/@babel/plugin-syntax-jsx/package.json	node-pkg	0
app/node_modules/@babel/plugin-syntax-logical-assignment-operators/package.json	node-pkg	0
app/node_modules/@babel/plugin-syntax-nullish-coalescing-operator/package.json	node-pkg	0
app/node_modules/@babel/plugin-syntax-numeric-separator/package.json	node-pkg	0
app/node_modules/@babel/plugin-syntax-object-rest-spread/package.json	node-pkg	0
app/node_modules/@babel/plugin-syntax-optional-catch-binding/package.json	node-pkg	0
app/node_modules/@babel/plugin-syntax-optional-chaining/package.json	node-pkg	0
app/node_modules/@babel/plugin-syntax-private-property-in-object/package.json	node-pkg	0
app/node_modules/@babel/plugin-syntax-top-level-await/package.json	node-pkg	0
app/node_modules/@babel/plugin-syntax-typescript/package.json	node-pkg	0
app/node_modules/@babel/template/package.json	node-pkg	0
app/node_modules/@babel/traverse/node_modules/debug/package.json	node-pkg	0
app/node_modules/@babel/traverse/node_modules/ms/package.json	node-pkg	0
app/node_modules/@babel/traverse/package.json	node-pkg	0
app/node_modules/@babel/types/package.json	node-pkg	0
app/node_modules/@bcoe/v8-coverage/dist/lib/package.json	node-pkg	0
app/node_modules/@bcoe/v8-coverage/package.json	node-pkg	0

- Análisis estático de código (SAST - Semgrep):
    - `reports/08_semgrep.json`
        - Semgrep detectó varias alertas relevantes de seguridad entre ellas:
            - ➢ Contenedores sin usuario no privilegiado en los Dockerfiles de "`ai-engine`", "`backend`" y "`web`".
            - ➢ Riesgo de path traversal en "`backend/src/controllers/evento.controller.ts`".
            - ➢ Recomendaciones de hardening en "`docker-compose.yml`", como "`no-new-privileges`" y "`read_only`".
            - ➢ Falta de atributo "`integrity`" en un recurso externo del frontend.
            - ➢ Formato inseguro: "`javascript.lang.security.audit.unsafe-formatstring`" en "`backend/seed-aparcamientos-finales.ts`"

10. CONCLUSIÓN

La evaluación de seguridad realizada mediante herramientas automáticas nos permitió identificar debilidades ,dependencias y algunos errores en la configuración de la aplicación. Los resultados obtenidos demuestran la importancia de integrar controles de seguridad desde fases tempranas del desarrollo. Además se comprobó mediante técnicas como SAST, SCA, análisis de contenedores y pruebas en ejecución nos ayudan a obtener una visión más completa de la seguridad del sistema de la aplicación FollowMe.